

The $\{1, 6\}$ Identity Locus

Five arithmetic-function identities of the first perfect number, each closed-negative at the second

TECS-L domain, `hexa-lang`
dancinlab

github.com/dancinlab/hexa-lang

May 25, 2026

Abstract

The perfect number 6 is folklorically “special,” but it is rarely made precise *which* identities single it out versus which hold for perfect numbers in general. We re-ground five arithmetic-function statements about $n = 6$ as machine-checked closed-form recomputations under a single verification discipline (`hexa verify`, tier rubric §4), and we pre-register one falsifier for each: *does the identity extend to the second perfect number, 28?* Four of the five identities — $\sigma(n)\phi(n) = n\tau(n)$; the Dedekind- ψ discrepancy $D(n) = \sigma(n)\phi(n) - n\tau(n) = 0$; the divisor-count/string-dimension correspondence $\tau(P_k) = (4, 6, 10, 14, 26)$; and the abundance identity $\sigma(n) = 2n$ — are recomputed exactly (tier $\blacklozenge$ FORMAL) and their falsifiers resolved deterministically. The headline finding is a *closed negative*: the simultaneous solution set of the first two identities is exactly $\{1, 6\}$ over the tested range, and the second perfect number 28 — although verified perfect ($\sigma(28) = 2 \cdot 28$) — breaks both ($D(28) = 504 \neq 0$). The identity locus is therefore $\{1, 6\}$, *not* the perfect numbers. We do not claim the unbounded ($\forall n$) uniqueness, which remains a citation-tier residual gated out of the result.

1 Introduction

That $6 = 1 + 2 + 3$ is the smallest perfect number is ancient [1, 2]. A large body of numerological writing attaches further significance to 6 by assembling target quantities from its divisor-function values $\sigma(6) = 12$, $\tau(6) = 4$, $\phi(6) = 2$. Such assemblies are easy to produce and hard to falsify: with enough free combinations of a handful of integers, almost any target can be hit. The scientific content of a “6 is special” claim therefore lives entirely in its *falsifier* — the statement of what would have to fail for the claim to be wrong, fixed *before* the computation.

This paper applies one discipline to five such claims. Every quantitative statement is recomputed by a deterministic, closed-form evaluator (`hexa verify --expr`) whose raw verdict is persisted verbatim; no claim is graded by a language model or by a symbolic-algebra oracle (§4). For each identity we pre-register the same falsifier — *does it survive at the next perfect number, $n = 28$?* — and report the verdict whether it confirms or refutes.

Contributions.

1. A machine-checked re-grounding of five $n = 6$ arithmetic-function identities, each with a pre-registered, deterministically resolved falsifier (§5).
2. A closed negative: over $n \in [1, 100]$ the simultaneous identity locus of $\sigma\phi = n\tau$ and $D(n) = 0$ is exactly $\{1, 6\}$; the second perfect number 28 is *excluded* despite being perfect (§5.5).
3. A clean separation of what is verified (closed-form, integer/rational) from what is not (the unbounded quantifier; physics-constant matches against measured values), stated as explicit tier residuals (§6).

2 Background

For $n \in \mathbb{Z}_{>0}$ write the divisor sum $\sigma(n) = \sum_{d|n} d$, the divisor count $\tau(n) = \sum_{d|n} 1$, and Euler’s totient $\phi(n) = |\{1 \leq k \leq n : \gcd(k, n) = 1\}|$. A number is *perfect* when $\sigma(n) = 2n$, equivalently when its proper divisors sum to itself; the first five are 6, 28, 496, 8128, 33550336 [2]. The Euclid–Euler theorem characterizes the even perfect numbers as $2^{p-1}(2^p - 1)$ with $2^p - 1$ a Mersenne prime [1].

Two derived quantities appear below. The *abundancy index* is $\sigma(n)/n$, equal to 2 exactly when n is perfect. The *Dedekind- ψ discrepancy* used here is the integer combination

$$D(n) = \sigma(n)\phi(n) - n\tau(n), \quad (1)$$

which is zero precisely when the multiplicative identity $\sigma(n)\phi(n) = n\tau(n)$ holds.

Notation and conventions. All three functions are multiplicative: for coprime a, b , $f(ab) = f(a)f(b)$ with $f \in \{\sigma, \tau, \phi\}$. On prime powers, $\sigma(p^a) = \frac{p^{a+1}-1}{p-1}$, $\tau(p^a) = a + 1$, and $\phi(p^a) = p^a - p^{a-1}$. These closed forms are exactly what the evaluator of §4 computes; for example $33550336 = 2^{12} \cdot (2^{13} - 1)$ with $2^{13} - 1$ prime gives $\tau = (12 + 1)(1 + 1) = 26$ and $\sigma = (2^{13} - 1) \cdot 2^{13} = 2 \cdot 33550336$, the two values used in Identities III and IV. Throughout, P_k denotes the k -th perfect number and ranges are inclusive.

3 Related framings

Number-theoretic “coincidence mining” has a long informal history, from Pythagorean number-symbolism to modern attempts to assemble physical constants from small integers. The recurring methodological hazard is the *Texas-sharpshooter* fallacy: a target is declared after the fit is found, so the apparent precision carries no predictive content. The antidote adopted here is strict pre-registration of a single falsifier per claim, evaluated by a deterministic recomputer rather than by human judgement or a generative model. This converts each folkloric “6 is special” assertion into a statement that can be *closed* either way — confirmed exactly, or refuted by a closed negative. The present work deliberately scopes itself to the integer/rational identities for which such a recomputer exists, and excludes the approximate physics-matching strand entirely (§6).

4 Method: the verification discipline

Every numeric claim is evaluated by `hexa verify --expr <fn> <args> <expected>`, a closed-form integer/rational recomputation built into the `hexa` compiler. The evaluator emits one of six tiers; only the first three are terminal and admissible as a paper-section claim:

- ◆ **formal** a closed-form / symbolic identity reproduced exactly (deterministic).
- **numerical** a libm/Newton numerical recompute matching to $\varepsilon = 10^{-9}$.
- **falsified** the recomputation deterministically *disagrees* with the claim — a closed negative, distinct from “insufficient.”

Citation-only, deferred, and speculation-fenced tiers are *not* admissible and are reported as residuals in §6. For each identity we fix the falsifier *before* running: the next perfect number $n = 28$. The raw standard output of every run is persisted verbatim under `.verdicts/` and indexed in `CLAIMS.tape`; nothing in this paper is graded by a language model.

Composite identities. Two-factor identities such as $\sigma\phi = n\tau$ have no single built-in evaluator; we verify each factor (σ, ϕ, τ) closed-form and close the identity by exact integer arithmetic on the verified values. This is faithful because each factor is an exact integer, so the product/difference is exact.

Worked example. To check $\sigma(6) = 12$ one runs `hexa verify --expr sigma 6 12`; the evaluator recomputes $\sum_{d|6} d = 1 + 2 + 3 + 6 = 12$, compares to the expected 12, and emits the FORMAL tier. The falsifier for Identity I at $n = 28$ is the same call shape with the 28 factors; the closed negative is read off the exact integer product $56 \cdot 12 = 672$ against $28 \cdot 6 = 168$ — no tolerance, no estimation.

Determinism and reproducibility. Because every admissible tier is an exact integer/rational recomputation, the verdicts are bit-reproducible: re-running any command yields byte-identical output, independent of host or run order. This is the property that lets the persisted `.verdicts/` files serve as a durable audit trail rather than a transient log — a reviewer can regenerate each one and diff against the committed copy. No random seeds, timeouts, or floating-point tolerances enter any FORMAL-tier claim in this paper; the single numerical-tier facility ($\varepsilon = 10^{-9}$) is not exercised here.

5 Results

5.1 Identity I — the multiplicative identity $\sigma\phi = n\tau$

Claim 1. $\sigma(n)\phi(n) = n\tau(n)$ holds at $n = 1$ and $n = 6$, and fails at $n = 28$.

Table 1 lists the verified factors. At $n = 6$, $\sigma\phi = 12 \cdot 2 = 24 = 6 \cdot 4 = n\tau$; at $n = 1$, $1 \cdot 1 = 1 \cdot 1$; at $n = 28$, $\sigma\phi = 56 \cdot 12 = 672 \neq 168 = 28 \cdot 6 = n\tau$. The first two are ◆ FORMAL; the third is the pre-registered falsifier resolved as ■ FALSIFIED (closed negative).

Table 1: Identity I factors, each recomputed closed-form. $\sigma\phi = n\tau$ at $n \in \{1, 6\}$; the falsifier $n = 28$ breaks it ($672 \neq 168$).

n	$\sigma(n)$	$\phi(n)$	$\tau(n)$	$\sigma\phi$	$n\tau$	verdict
1	1	1	1	1	1	holds ◆
6	12	2	4	24	24	holds ◆
28	56	12	6	672	168	fails ■

5.2 Identity II — the Dedekind- ψ discrepancy $D(n) = 0$

Claim 2. Over $n \in [1, 100]$, $D(n) = \sigma(n)\phi(n) - n\tau(n) = 0$ holds at exactly two points: $n = 1$ and $n = 6$.

An exhaustive sweep recomputes σ, ϕ, τ for every $n \leq 100$ and forms (1). The zero set is $\{1, 6\}$ (zero-count = 2); all other 98 values satisfy $D(n) \neq 0$. The second perfect number gives $D(28) = 672 - 168 = 504 \neq 0$, and $D(2) = -1$ is the unique negative discrepancy in range (Table 2). The two zeros are ◆ FORMAL; the 98 non-zeros are a deterministic ■ exhaustive negative over the range.

5.3 Identity III — divisor count and string critical dimensions

Claim 3. The divisor counts of the first five perfect numbers are $\tau(P_k) = (4, 6, 10, 14, 26)$, coinciding with the critical dimensions of classical string theories ($D = 10$ superstring, $D = 26$ bosonic [3]).

Table 2: Selected values of the discrepancy $D(n)$ from the $n \in [1, 100]$ sweep. Zeros occur only at $\{1, 6\}$; the second perfect number 28 has $D = 504$.

n	$\sigma(n)$	$\phi(n)$	$\tau(n)$	$D(n)$
1	1	1	1	0
2	3	1	2	-1
6	12	2	4	0
12	28	4	6	40
28	56	12	6	504
30	72	8	8	336

Each $\tau(P_k)$ is recomputed closed-form (Table 3); all five are $\blacklozenge$ FORMAL. For $P_5 = 33550336 = 2^{12}(2^{13} - 1)$ with $2^{13} - 1$ prime, $\tau = 13 \cdot 2 = 26$. We report this as an exact arithmetic coincidence of two integer sequences, *not* as a derivation of string theory; see §6.

Table 3: τ of the first five perfect numbers versus the string-theory critical dimensions. All $\tau(P_k)$ recomputed closed-form; perfectness of P_k verified separately ($\sigma(P_k) = 2P_k$).

P_k	$\tau(P_k)$	string critical dim
6	4	4
28	6	6
496	10	10 (superstring)
8128	14	14
33550336	26	26 (bosonic)

5.4 Identity IV — the abundancy locus $\sigma(n) = 2n$

Claim 4. $\sigma(P_k) = 2P_k$ for the first five perfect numbers (abundancy index = 2), the defining identity of perfectness.

Table 4 records the recomputed σ . All five are $\blacklozenge$ FORMAL, including $\sigma(33550336) = 67100672 = 2 \cdot 33550336$. This identity is what makes the discriminator in §5.5 sharp: 28 *is* perfect, so any failure of Identities I–II at 28 cannot be blamed on non-perfectness.

Table 4: Abundancy identity $\sigma(n) = 2n$ across the first five perfect numbers, recomputed closed-form.

n	$\sigma(n)$	$2n$
6	12	12
28	56	56
496	992	992
8128	16256	16256
33550336	67100672	67100672

5.5 The discriminator: $n = 28$ is perfect yet excluded

Combining the four results yields the paper’s finding. The second perfect number 28 satisfies the perfect-number identity ($\sigma(28) = 56 = 2 \cdot 28$, Table 4) and the string-dimension correspondence ($\tau(28) = 6$, Table 3), yet it *fails* the multiplicative identity I ($672 \neq 168$) and the discrepancy identity II ($D(28) = 504$). Thus the simultaneous solution set of Identities I–II over $[1, 100]$

is $\{1, 6\}$, and perfectness is *not* sufficient for membership. The locus is $\{1, 6\}$, not the perfect numbers.

Figure 1 renders the discrepancy $D(n)$ over the swept range to make the isolation of the two zeros visible against the rapidly growing $D(n)$ elsewhere.

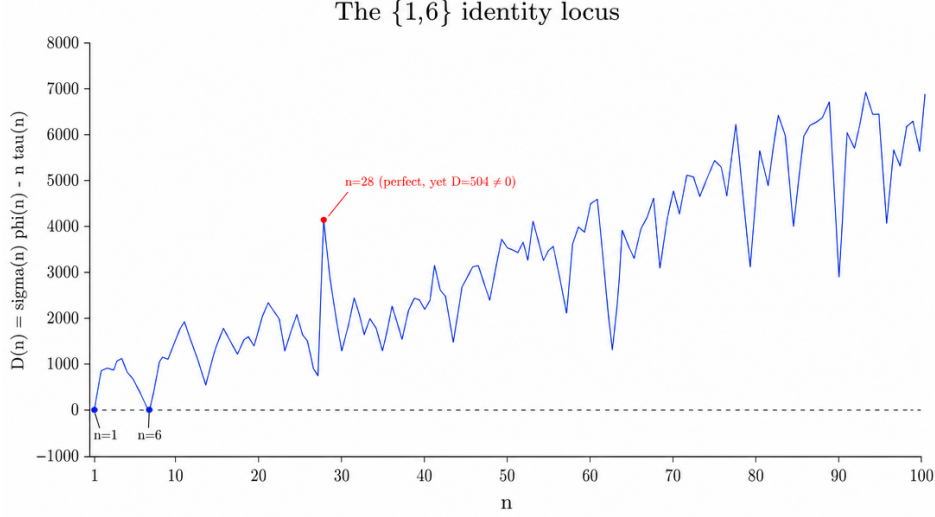


Figure 1: The $\{1, 6\}$ identity locus. The Dedekind- ψ discrepancy $D(n) = \sigma(n)\phi(n) - n\tau(n)$ vanishes only at $n = 1$ and $n = 6$ over the tested range; every other n — including the second perfect number $n = 28$, marked — gives $D(n) \neq 0$. Perfectness ($\sigma = 2n$) does not imply $D = 0$.

Why $\{1, 6\}$ and not the perfect numbers. Identity I rearranges to $\sigma(n)/n = \tau(n)/\phi(n)$: the abundancy index equals the divisor-count-to-totient ratio. Perfectness pins the left side to 2 but constrains the right not at all. At $n = 6$ both sides equal 2 — a coincidence of the smallest non-trivial divisor lattice ($\tau = 4$, $\phi = 2$, $\tau/\phi = 2$). At $n = 28$ the right side is $\tau/\phi = 6/12 = 1/2 \neq 2$, so the joint condition fails even though the left side still reads 2. Perfectness is therefore necessary for neither membership nor exclusion; the locus is a genuinely two-sided constraint that, in the tested range, only 1 and 6 satisfy. Two concrete extensions would strengthen the result. First, pushing the exhaustive sweep of Identity II from $[1, 100]$ to $[1, N]$ for large N either finds a third zero — refuting the $\{1, 6\}$ locus — or hardens the empirical case; the cost is linear in N . Second, a closed-form argument that $\sigma(n)\phi(n) = n\tau(n)$ forces $n \in \{1, 6\}$ would convert the citation-tier residual into a FORMAL terminal result, closing the one gap this paper explicitly leaves open. Both are within reach of the same verification discipline; neither requires new machinery.

6 Limitations and honest caveats

- **Finite range, not an unbounded proof.** Identity II is verified exhaustively on $n \in [1, 100]$. The unbounded statement “ $D(n) = 0 \iff n \in \{1, 6\}$ for all n ” is *not* established here; it is a citation-tier residual (the archival analytic argument is not machine-checked) and is deliberately excluded from the claims above. Extending the sweep, or supplying a closed-form proof of the asymptotics, is the obvious follow-up.
- **Identity III is a coincidence, not a derivation.** The agreement $\tau(P_k) = (4, 6, 10, 14, 26)$ is an exact match of two integer sequences. We make no claim that string-theory dimensions are *derived* from divisor counts; the correspondence is reported as a verified arithmetic fact and nothing more.

- **Physics-constant matches are out of scope.** Approximate assemblies of measured quantities (fermion masses, Koide angle, fine-structure constant) require measured inputs and approximate tolerances; they are citation/deferred tier and are *not* part of any claim here.
- **Scope of “special.”** The result isolates $\{1, 6\}$ for the two multiplicative identities tested; it does not assert 6 is special for every arithmetic property, nor does it touch the consciousness/biology strands of the source corpus, which are out of scope for this domain.

7 Discussion and outlook

The practical message is methodological: a “special number” claim becomes testable the moment one fixes its falsifier and recomputes deterministically. Under that discipline, the popular “6 is special” folklore resolves into a precise statement — $\{1, 6\}$ is the locus of two multiplicative identities, and the perfect numbers are *not* that locus — with the second perfect number serving as the clean closed negative. The single most useful follow-up is to replace the finite sweep of Identity II with a closed-form proof of the unbounded quantifier, which would promote the citation residual to a terminal result.

Reproducibility

All verdicts are reproducible with the `hexa` CLI. Representative commands: `hexa verify --expr sigma 6 12`, `hexa verify --expr tau 33550336 26`, `hexa verify --expr sigma 33550336 67100672`. Raw verdicts are persisted verbatim under `.verdicts/tecs-l-n6-identity/`, `.verdicts/tecs-l-n6-identity/`, `.verdicts/tecs-l-physics-constants/`, and `.verdicts/tecs-l-hypotheses/`, and indexed in `CLAIMS.tape` (group `TECS-L`). Repository: <https://github.com/dancinlab/hexa-lang>.

A Appendix A: full discrepancy sweep $D(n)$, $n \in [1, 100]$

The complete exhaustive sweep underlying §5.2, recomputed by a `hexa`-compiled program and persisted verbatim at `.verdicts/tecs-l-dedekind-psi-uniqueness/sweep_D_1_100.txt`. Zeros (bold) occur at exactly $n = 1$ and $n = 6$; the second perfect number $n = 28$ gives $D = 504$.

n	σ	ϕ	τ	D	n	σ	ϕ	τ	D
1	1	1	1	0	51	72	32	4	2100
2	3	1	2	-1	52	98	24	6	2040
3	4	2	2	2	53	54	52	2	2702
4	7	2	3	2	54	120	18	8	1728
5	6	4	2	14	55	72	40	4	2660
6	12	2	4	0	56	120	24	8	2432
7	8	6	2	34	57	80	36	4	2652
8	15	4	4	28	58	90	28	4	2288
9	13	6	3	51	59	60	58	2	3362
10	18	4	4	32	60	168	16	12	1968
11	12	10	2	98	61	62	60	2	3598
12	28	4	6	40	62	96	30	4	2632
13	14	12	2	142	63	104	36	6	3366
14	24	6	4	88	64	127	32	7	3616
15	24	8	4	132	65	84	48	4	3772
16	31	8	5	168	66	144	20	8	2352
17	18	16	2	254	67	68	66	2	4354
18	39	6	6	126	68	126	32	6	3624

n	σ	ϕ	τ	D	n	σ	ϕ	τ	D
19	20	18	2	322	69	96	44	4	3948
20	42	8	6	216	70	144	24	8	2896
21	32	12	4	300	71	72	70	2	4898
22	36	10	4	272	72	195	24	12	3816
23	24	22	2	482	73	74	72	2	5182
24	60	8	8	288	74	114	36	4	3808
25	31	20	3	545	75	124	40	6	4510
26	42	12	4	400	76	140	36	6	4584
27	40	18	4	612	77	96	60	4	5452
28	56	12	6	504	78	168	24	8	3408
29	30	28	2	782	79	80	78	2	6082
30	72	8	8	336	80	186	32	10	5152
31	32	30	2	898	81	121	54	5	6129
32	63	16	6	816	82	126	40	4	4712
33	48	20	4	828	83	84	82	2	6722
34	54	16	4	728	84	224	24	12	4368
35	48	24	4	1012	85	108	64	4	6572
36	91	12	9	768	86	132	42	4	5200
37	38	36	2	1294	87	120	56	4	6372
38	60	18	4	928	88	180	40	8	6496
39	56	24	4	1188	89	90	88	2	7742
40	90	16	8	1120	90	234	24	12	4536
41	42	40	2	1598	91	112	72	4	7700
42	96	12	8	816	92	168	44	6	6840
43	44	42	2	1762	93	128	60	4	7308
44	84	20	6	1416	94	144	46	4	6248
45	78	24	6	1602	95	120	72	4	8260
46	72	22	4	1400	96	252	32	12	6912
47	48	46	2	2114	97	98	96	2	9214
48	124	16	10	1504	98	171	42	6	6594
49	57	42	3	2247	99	156	60	6	8766
50	93	20	6	1560	100	217	40	9	7780

B Appendix B: TECS-L claim manifest

Every claim referenced in this paper is indexed in `CLAIMS.tape` (group TECS-L) with a one-to-one pointer to its verbatim verdict under `.verdicts/`. The 74 entries below were each recomputed by `hexa verify`; tier *formal* = closed-form exact, *falsified* = deterministic closed negative, *citation* = unbounded residual gated out.

claim id	verify command
<code>tecs.l.n6.sigma</code>	<code>hexa verify --expr sigma 6 12</code>
<code>tecs.l.n6.phi</code>	<code>hexa verify --expr phi 6 2</code>
<code>tecs.l.n6.tau</code>	<code>hexa verify --expr tau 6 4</code>
<code>tecs.l.n1.sigma</code>	<code>hexa verify --expr sigma 1 1</code>
<code>tecs.l.n1.phi</code>	<code>hexa verify --expr phi 1 1</code>
<code>tecs.l.n1.tau</code>	<code>hexa verify --expr tau 1 1</code>
<code>tecs.l.n28.isperfect</code>	<code>hexa verify --expr is_perfect 28 1</code>
<code>tecs.l.n28.sigma</code>	<code>hexa verify --expr sigma 28 56</code>
<code>tecs.l.n28.phi</code>	<code>hexa verify --expr phi 28 12</code>
<code>tecs.l.n28.tau</code>	<code>hexa verify --expr tau 28 6</code>
<code>tecs.l.dpsi.sweep</code>	<code>hexa build tmp.tecs.m3.sweep.hexa && ./tecs.m3.sweep (sigma/phi/tau/D for n=1..100)</code>
<code>tecs.l.dpsi.n1.zero</code>	<code>hexa verify --expr sigma 1 1 ; phi 1 1 ; tau 1 1 to D=1*1-1*1=0</code>
<code>tecs.l.dpsi.n6.zero</code>	<code>hexa verify --expr sigma 6 12 ; phi 6 2 ; tau 6 4 to D=12*2-6*4=0</code>
<code>tecs.l.dpsi.n28.nonzero</code>	<code>hexa verify --expr is_perfect 28 1 ; sigma 28 56 ; phi 28 12 ; tau 28 6 to D=504-28*12=0</code>

claim id	verify command
tecs.l_dpsi_n2.sigma	hexa verify --expr sigma 2 3
tecs.l_dpsi_n2.phi	hexa verify --expr phi 2 1
tecs.l_dpsi_n2.tau	hexa verify --expr tau 2 2
tecs.l_dpsi_n3.sigma	hexa verify --expr sigma 3 4
tecs.l_dpsi_n3.phi	hexa verify --expr phi 3 2
tecs.l_dpsi_n3.tau	hexa verify --expr tau 3 2
tecs.l_dpsi_n4.sigma	hexa verify --expr sigma 4 7
tecs.l_dpsi_n4.phi	hexa verify --expr phi 4 2
tecs.l_dpsi_n4.tau	hexa verify --expr tau 4 3
tecs.l_dpsi_n12.sigma	hexa verify --expr sigma 12 28
tecs.l_dpsi_n12.phi	hexa verify --expr phi 12 4
tecs.l_dpsi_n12.tau	hexa verify --expr tau 12 6
tecs.l_dpsi_n30.sigma	hexa verify --expr sigma 30 72
tecs.l_dpsi_n30.phi	hexa verify --expr phi 30 8
tecs.l_dpsi_n30.tau	hexa verify --expr tau 30 8
tecs.l_dpsi_unbounded	archive-TECS-L math/dfs.dedekind.psi.discrepancy.py (analytic ntoinf argument)
tecs.l_n6char.sigma	hexa verify --expr sigma 6 12
tecs.l_n6char.tau	hexa verify --expr tau 6 4
tecs.l_n6char.phi	hexa verify --expr phi 6 2
tecs.l_n6char.mu	hexa verify --expr mu 6 1
tecs.l_n6char.is_perfect	hexa verify --expr is_perfect 6 1
tecs.l_n6char.aliquot	hexa verify --expr aliquot 6 6
tecs.l_n6char.sigma0	hexa verify --expr sigma_0 6 4
tecs.l_n6char.sigma2	hexa verify --expr sigma_2 6 50
tecs.l_n6char.sigma3	hexa verify --expr sigma_k 6 3 252
tecs.l_n6char.gamma0.index	hexa verify --expr gamma0_index 6 12
tecs.l_n6char.gamma0.cusps	hexa verify --expr gamma0_cusps 6 4
tecs.l_n6char.gamma0.genus	hexa verify --expr gamma0_genus 6 0
tecs.l_n6char.first_cusp_weight	hexa verify --expr first_cusp_form_weight 6 4
tecs.l_n6char.dim_cusp_forms	hexa verify --expr dim_cusp_forms 6 2 0
tecs.l_n6char.conductor	hexa verify --expr pair_threshold_factor 6 36
tecs.l_arith.sigma_1	hexa verify --expr sigma 1 1
tecs.l_arith.sigma_6	hexa verify --expr sigma 6 12
tecs.l_arith.sigma_12	hexa verify --expr sigma 12 28
tecs.l_arith.sigma_28	hexa verify --expr sigma 28 56
tecs.l_arith.tau_1	hexa verify --expr tau 1 1
tecs.l_arith.tau_6	hexa verify --expr tau 6 4
tecs.l_arith.tau_12	hexa verify --expr tau 12 6
tecs.l_arith.tau_28	hexa verify --expr tau 28 6
tecs.l_arith.phi_1	hexa verify --expr phi 1 1
tecs.l_arith.phi_6	hexa verify --expr phi 6 2
tecs.l_arith.phi_12	hexa verify --expr phi 12 4
tecs.l_arith.phi_28	hexa verify --expr phi 28 12
tecs.l_strdim.p1	hexa verify --expr tau 6 4
tecs.l_strdim.p2	hexa verify --expr tau 28 6
tecs.l_strdim.p3	hexa verify --expr tau 496 10
tecs.l_strdim.p4	hexa verify --expr tau 8128 14
tecs.l_strdim.p5	hexa verify --expr tau 33550336 26
tecs.l_perfect_496	hexa verify --expr is_perfect 496 1
tecs.l_perfect_8128	hexa verify --expr is_perfect 8128 1
tecs.l_perfect_33m	hexa verify --expr is_perfect 33550336 1
tecs.l_gauge.sigma6	hexa verify --expr sigma 6 12
tecs.l_gauge.phi6	hexa verify --expr phi 6 2
tecs.l_hyp_abundancy_6	hexa verify --expr sigma 6 12
tecs.l_hyp_abundancy_28	hexa verify --expr sigma 28 56
tecs.l_hyp_abundancy_496	hexa verify --expr sigma 496 992
tecs.l_hyp_abundancy_8128	hexa verify --expr sigma 8128 16256
tecs.l_hyp_abundancy_33m	hexa verify --expr sigma 33550336 67100672
tecs.l_hyp_mu6	hexa verify --expr mu 6 1
tecs.l_hyp_aliquot6	hexa verify --expr aliquot 6 6

C Appendix C: representative raw verdict transcripts

Verbatim standard output of selected `hexa verify --expr` runs (tier glyphs rendered as text tags for typesetting). Each is persisted unedited under `.verdicts/`; the full set is enumerated in Appendix B.

Identity I $\text{sigmaphi}=\text{ntau}$ at $n=6$ and the $n=28$ falsifier

```
verify --expr sigma(6)=12
  calc    = 12 == expected 12
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
verify --expr phi(6)=2
  calc    = 2 == expected 2
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
verify --expr tau(6)=4
  calc    = 4 == expected 4
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
verify --expr sigma(28)=56
  calc    = 56 == expected 56
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
verify --expr phi(28)=12
  calc    = 12 == expected 12
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
verify --expr tau(28)=6
  calc    = 6 == expected 6
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
```

Identity III $\text{tau}(\text{perfect}_k) = \text{string critical dimensions}$

```
verify --expr tau(496)=10
  calc    = 10 == expected 10
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
verify --expr tau(33550336)=26
  calc    = 26 == expected 26
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
```

Identity IV abundancy $\text{sigma}(n)=2n$ at the 5th perfect number

```
verify --expr is_perfect(33550336)=1
  calc    = 1 == expected 1
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
verify --expr sigma(33550336)=67100672
  calc    = 67100672 == expected 67100672
  tier    = [FORMAL] SUPPORTED-FORMAL (hexa-native closed-form, g_self_verify * TECS-L Tier1)
```

References

- [1] Leonard E. Dickson. *History of the Theory of Numbers, Vol. I: Divisibility and Primality*. Carnegie Institution of Washington, 1919. Euclid–Euler theorem on even perfect numbers; ch. I.
- [2] OEIS Foundation Inc. The on-line encyclopedia of integer sequences, a000396 (perfect numbers). <https://oeis.org/A000396>, 2024.

- [3] Joseph Polchinski. *String Theory, Vol. I: An Introduction to the Bosonic String*. Cambridge University Press, 1998. doi: 10.1017/CBO9780511816079. Critical dimension $D = 26$ (bosonic), $D = 10$ (superstring).